ChatGPT

This is a copy of a conversation between ChatGPT & Anonymous.
Report conversation

Uploaded a file

I think we're using the following git for utilities and environment (https://github.com/liuyiqiandrew/AuctionNet/tree/main)

We would use PPO for baseline for doing modifications and it is being implemented in the ppo branch. Then, what would be a good division of work to realize our proposal for the 5 member team.

I'm checking the repo layout and your proposal together so the division of work matches the actual code surface rather than being generic. Then I'll turn that into a concrete 5-person split with milestones and interfaces.

I've got enough of the repo shape now. The good news is your current branch layout already suggests a clean split: one person on the PPO spine, two on modifications, one on data/infra, one on evaluation/writing, instead of five people each inventing separate agents.

One important thing I found: the current PPO branch already implements parts of two proposed "modifications." It already has offline BC warm-start plus a terminal CPA penalty in training, and the state already includes time left, budget left, recent history summaries, and traffic-volume features. So the team should not divide work as if reward shaping and budget-awareness are still blank slates.

Thought for 2m 57s

Yes — with 5 people, I would **not** split by "five separate algorithms." Given your proposal's stated plan of first getting a working baseline, then parallelizing a **small number of targeted PPO modifications**, the cleanest setup is:

2 backbone roles + 2 modification roles + 1 evaluation/integration role. That matches both the proposal and the repo structure, which already separates data processing,

baseline training, strategy wrappers, and evaluation. RL_proposal_v5 GitHub +1

A good division is this:

1. **Baseline PPO owner**

   This person owns getting `ppo` to train end-to-end and produce a model that can actually be evaluated cleanly. In your branch, `main/main_ppo.py` just calls `run/run_ppo.py`, and `run_ppo.py` already contains the main PPO training loop, offline warm-start, rollout collection, and model saving. So this person should stabilize that loop, verify reward/value losses behave sensibly, manage checkpoints, and make sure the saved JIT policy is valid. They should be your strongest implementation/debugging person. GitHub +2

2. **Data / infra / reproducibility owner**

   This person owns everything needed so the rest of the team can run experiments instead of fighting the environment: dataset prep, path consistency, cluster scripts, config management, seed control, and logging. AuctionNet's training side explicitly depends on the data conversion pipeline in `train_data_generator`, and your proposal already flags baseline reproduction and dataset scale as early engineering bottlenecks. This person should make a one-command workflow for "process data → train PPO → evaluate PPO," plus a short README for the team. RL_proposal_v5 GitHub +1

3. **Budget-aware state / pacing modification owner**

   This person should extend the **state representation and action logic**, not reimplement PPO. Your current PPO branch already uses a shared `state_builder.py`, and that state already includes time left, budget left, historical bid / winning-cost / conversion summaries, and traffic-volume features. So the right job here is to test *better* budget pacing features: spend-so-far ratio, expected spend-to-go, recent cost-per-conversion proxy, budget depletion rate, and maybe action rescaling tied to remaining budget. This is a very natural "domain-specific" mod because it stays close to your proposal and plugs directly into an existing interface. RL_proposal_v5 GitHub +1

4. **Temporal / constraint-handling modification owner**

I would give one person the second substantive algorithmic branch: either **better reward shaping / constraint handling** or **temporal modeling**. One reason this split is good is that your current PPO already has a **behavior-cloning warm start** and a **terminal CPA penalty**, so "reward shaping" is not blank-slate work anymore; this person should turn that into a real ablation study, e.g. terminal-only penalty vs denser shaping vs a simple Lagrangian-style penalty. If they move fast, the stretch goal is a lightweight recurrent policy or richer temporal encoder. In a 5-week course project, I would start with constraint shaping first and only upgrade to recurrent PPO if the baseline is solid by the end of week 2. RL_proposal_v5 GitHub +1

5. **Evaluation / ablation / integration owner**

   This person should own the question: "Do we actually have evidence that the modification helped?" In your repo, training and inference are separated through the unified bidding strategy interface, and right now `strategy/__init__.py` still points to IQL while PPO is commented out. So this person should wire PPO cleanly into evaluation, standardize metrics, build plotting scripts/tables, and run the ablation matrix. They should also explicitly check train/eval mismatch: your PPO training controller uses a simplified opponent roster built from PID and OnlineLP strategies so rollouts do not depend on loading pretrained checkpoints, which is convenient for training but means evaluation fidelity needs to be checked carefully. GitHub +2

I would run the project in **three phases**:

Phase 1: everybody helps baseline reproduction.
For the first 7–10 days, roles 1 and 2 lead, but everyone helps get a clean PPO run, one saved model, one offline eval, and one online smoke test. Do not parallelize modifications before that. That is exactly consistent with your proposal's "first milestone is a working end-to-end baseline." RL_proposal_v5

Phase 2: parallel modifications.
Once baseline PPO is stable, roles 3 and 4 branch off onto two modifications, while role 5 locks down eval scripts and role 2 keeps runs reproducible. This is where you get your real project content: baseline PPO, PPO + budget-state, PPO + constraint shaping, and maybe PPO + both. RL_proposal_v5

Phase 3: integration and paper-ready ablations.
In the last stretch, everyone converges on results and writing. Role 5 should drive the table/figure structure, but roles 3 and 4 each need to write the methods/results

subsection for their mod, and role 1 should write the baseline/training details. That avoids the classic failure mode where only one person “does the writeup.”

RL_proposal_v5

If you want the most practical version, I would define the team deliverables like this:

- **Person 1:** reproducible PPO baseline result
- **Person 2:** reproducible training/eval pipeline + cluster scripts
- **Person 3:** budget-aware state / pacing ablation
- **Person 4:** reward-shaping or constraint-handling ablation
- **Person 5:** evaluation harness, plots, result tables, final integration

And I would make the **official fallback plan** this:

- If recurrent PPO is unstable, replace it with **history-feature PPO** rather than forcing an RNN.
- If online evaluation is too slow, keep online results as a final check and rely on offline ablations for most iteration.
- If only one modification works, submit **baseline PPO + one strong, well-ablated modification** rather than three half-working ideas.

That is probably the highest-probability path to a solid course project.

If you want, I can turn this into a **team task board with week-by-week milestones and branch names**.

Sources

Sources

I think here's the thing, we have PPO and IQL as baseline, we can do reward shaping and two other modifications proposed in the proposal. Can you give five extentions in total that can improve PPO / IQL performance that we can divide word such that each person can implement one.

I’m mapping this to a division that is actually parallelizable. The main constraint is not “five cool ideas,” it’s “five ideas that do not all require rewriting the same PPO/IQL core

at once."

I found the main design constraint in your fork: the current PPO branch already includes a BC warm-start, a terminal CPA penalty, and a 16-dim state with budget/time/history/traffic summaries, while the default evaluation interface still points to IQL. So the "five extensions" should be defined as stronger next steps, not as if those ideas are still untouched. GitHub +2

Thought for 1m 34s

Yes. Given your proposal and the current fork, the cleanest 5-way split is:

- keep **PPO and IQL as fixed baselines**
- define **5 extensions** that touch mostly different parts of the stack
- make each extension ablatable against the same eval harness

Also, because the current PPO branch already has a **BC warm-start**, a **terminal CPA penalty**, and a **16 GitHub +2 *, the extension versions should be the "next level" of those ideas, not the basic versions. GitHub +2 already motivates reward shaping, recurrent/history-aware policies, and budget-aware design in AuctionNet's sparse, delayed, competitive setting. RL_proposal_v5 5 I would use.

1. **Constraint-aware dense reward shaping**

   Upgrade the current PPO terminal CPA penalty into a real constrained-reward design. Instead of only penalizing CPA violation at episode end, add per-step shaping terms tied to conversions, spend efficiency, pacing error, and constraint violation. For IQL, this means recomputing rewards in the offline dataset; for PPO, it means changing the rollout reward path. This is one of the highest-probability improvements because the environment has sparse/delayed signal, and your current PPO only uses a final CPA overrun penalty. GitHub one person works mostly in reward construction and training config.
   Risk: low to medium.
   Why it parallelizes well: it does not require changing the policy architecture.

2. **Budget-aware pacing features plus action gating**

   Your current state already includes time left, budget left, and coarse recent-history statistics. The extension is to add stronger pacing signals: spend-so-far ratio, target-vs-actual spend gap, rolling win rate, rolling conversion rate, estimated future traffic pressure, and recent market price level. Then add a lightweight action post-processor that scales bids down when the agent is overspending relative to

time left. This is better framed as **budget pacing control**, not just "budget-aware state." The current 16-dim shared state is a good starting point, but it is still quite compressed. GitHub one person on state/action engineering.
Risk: low to medium.
Why it parallelizes well: mostly isolated to `state_builder` and strategy wrapper logic.

3. **Temporal / recurrent policy or sequence encoder**

   Your proposal already points toward recurrent policies for partial observability. RL_proposal_v5 state summarizes history using means and "last three" aggregates, which is simple but lossy. GitHub is to replace or augment that with a short-horizon GRU/LSTM/TCN encoder over recent ticks. For PPO this becomes a recurrent actor-critic or sequence-conditioned policy; for IQL it becomes an encoder over offline trajectory windows before the Q/policy heads.

   Owner: one stronger modeling person.
   Risk: medium to high.
   Why it parallelizes well: it touches the model, but not reward construction or data processing much.

4. **Offline data curation / curriculum / replay reweighting**

   This is the best "new" extension beyond your original three. AuctionNet provides logged trajectories from many agents and a large offline dataset, so you can likely gain a lot by changing *which data the learner emphasizes*, not only the architecture. The extension can be: oversample better-performing trajectories, reweight by conversion-bearing segments, stratify by budget regime, or run a curriculum from easier/stable slices to harder ones. For PPO, this naturally plugs into the existing BC warm-start; for IQL, it changes the offline sampler directly. The repo's benchmark explicitly includes an 80GB pre-generated dataset with diverse agents, so this is a very natural lever. GitHub one person on dataset loader / sampler / preprocessing.
   Risk: medium.
   Why it parallelizes well: mostly independent of policy-core code.

5. **Market-regime / opponent-aware conditioning**

   This is the other strong "new" extension. Your proposal emphasizes that the environment is competitive and effectively nonstationary. RL_proposal_v5 etending one stationary policy is always enough, add a context signal for auction regime:

recent least-winning-cost level, volatility, win-rate trend, exposure trend, conversion trend, traffic intensity, maybe a simple clustered "market regime ID." Then condition PPO/IQL on that regime, or use a lightweight mixture-of-experts head. This is more realistic than full explicit opponent modeling and is still implementable in a course project.

Owner: one person on context features / conditioning head.
Risk: medium.
Why it parallelizes well: conceptually separate from reward shaping and sequence modeling.

I would assign them like this:

- **Person 1:** dense reward shaping / constrained objective
- **Person 2:** budget pacing state + action gating
- **Person 3:** temporal/recurrent encoder
- **Person 4:** offline data curation / replay weighting / curriculum
- **Person 5:** market-regime conditioning

That is a much better split than "one person owns PPO, one owns IQL, one owns reward shaping…" because otherwise several people will bottleneck on the same training files.

My strongest recommendation is to define the project around this ablation grid:

- PPO baseline
- IQL baseline
- PPO + each relevant extension
- IQL + each relevant extension where feasible
- maybe one combined model at the end

In practice, not every extension has to be implemented on both baselines. A realistic mapping is:

- **both PPO and IQL:** reward shaping, budget pacing, regime conditioning
- **primarily PPO:** recurrent/temporal policy
- **primarily IQL, but also PPO warm-start:** offline data curation

If you want the highest-probability subset, I would prioritize these three first:
reward shaping, budget pacing, offline data curation.
Those are the most likely to yield measurable gains without destabilizing everything.

The riskiest but potentially most interesting one is:
temporal/recurrent modeling.

The most novel “auction-specific” one is:
market-regime conditioning.

I can turn this into a concrete **person-by-person task sheet with branch names, touched files, and week-by-week milestones**.

Sources

Sources